```
pip install spacy transformers torch pandas
```

```
Requirement already satisfied: spacy in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: transformers in /usr/local/lib/python3.12/dist-pa
Requirement already satisfied: torch in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: pandas in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: spacy-legacy<3.1.0,>=3.0.11 in /usr/local/lib/pyt
Requirement already satisfied: spacy-loggers<2.0.0,>=1.0.0 in /usr/local/lib/pyt
Requirement already satisfied: murmurhash<1.1.0,>=0.28.0 in /usr/local/lib/pytho
Requirement already satisfied: cymem<2.1.0,>=2.0.2 in /usr/local/lib/python3.12/
Requirement already satisfied: preshed<3.1.0,>=3.0.2 in /usr/local/lib/python3.1
Requirement already satisfied: thinc<8.4.0,>=8.3.4 in /usr/local/lib/python3.12/
Requirement already satisfied: wasabi<1.2.0,>=0.9.1 in /usr/local/lib/python3.12
Requirement already satisfied: srsly<3.0.0,>=2.4.3 in /usr/local/lib/python3.12/
Requirement already satisfied: catalogue<2.1.0,>=2.0.6 in /usr/local/lib/python3
Requirement already satisfied: weasel<0.5.0,>=0.4.2 in /usr/local/lib/python3.12
Requirement already satisfied: typer-slim<1.0.0,>=0.3.0 in /usr/local/lib/python
Requirement already satisfied: tqdm<5.0.0,>=4.38.0 in /usr/local/lib/python3.12/
Requirement already satisfied: numpy>=1.19.0 in /usr/local/lib/python3.12/dist-p
Requirement already satisfied: requests<3.0.0,>=2.13.0 in /usr/local/lib/python3
Requirement already satisfied: pydantic!=1.8,!=1.8.1,<3.0.0,>=1.7.4 in /usr/loca
Requirement already satisfied: jinja2 in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: setuptools in /usr/local/lib/python3.12/dist-pack
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.12/dist
Requirement already satisfied: filelock in /usr/local/lib/python3.12/dist-packag
Requirement already satisfied: huggingface-hub<2.0,>=1.3.0 in /usr/local/lib/pyt
Requirement already satisfied: pyyaml>=5.1 in /usr/local/lib/python3.12/dist-pac
Requirement already satisfied: regex!=2019.12.17 in /usr/local/lib/python3.12/di
Requirement already satisfied: tokenizers<=0.23.0,>=0.22.0 in /usr/local/lib/pyt
Requirement already satisfied: safetensors>=0.4.3 in /usr/local/lib/python3.12/d
Requirement already satisfied: typing-extensions>=4.10.0 in /usr/local/lib/pytho
Requirement already satisfied: sympy>=1.13.3 in /usr/local/lib/python3.12/dist-p
Requirement already satisfied: networkx>=2.5.1 in /usr/local/lib/python3.12/dist
Requirement already satisfied: fsspec>=0.8.5 in /usr/local/lib/python3.12/dist-p
Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python3.
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.12/dist-pa
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.12/dist-
Requirement already satisfied: hf-xet<2.0.0,>=1.2.0 in /usr/local/lib/python3.12
Requirement already satisfied: httpx<1,>=0.23.0 in /usr/local/lib/python3.12/dis
Requirement already satisfied: shellingham in /usr/local/lib/python3.12/dist-pac
Requirement already satisfied: annotated-types>=0.6.0 in /usr/local/lib/python3.
Requirement already satisfied: pydantic-core==2.41.4 in /usr/local/lib/python3.1
Requirement already satisfied: typing-inspection>=0.4.2 in /usr/local/lib/python
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packag
Requirement already satisfied: charset_normalizer<4,>=2 in /usr/local/lib/python
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.12/dist-pa
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.12/d
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.12/d
Requirement already satisfied: mpmath<1.4,>=1.1.0 in /usr/local/lib/python3.12/d
Requirement already satisfied: blis<1.4.0,>=1.3.0 in /usr/local/lib/python3.12/d
Requirement already satisfied: confection<1.0.0,>=0.0.1 in /usr/local/lib/python
Requirement already satisfied: typer>=0.24.0 in /usr/local/lib/python3.12/dist-p
Requirement already satisfied: cloudpathlib<1.0.0,>=0.7.0 in /usr/local/lib/pyth
Requirement already satisfied: smart-open<8.0.0,>=5.2.1 in /usr/local/lib/python
Requirement already satisfied: MarkupSafe>=2.0 in /usr/local/lib/python3.12/dist
Requirement already satisfied: anyio in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: httpcore==1.* in /usr/local/lib/python3.12/dist-p
Requirement already satisfied: h11>=0.16 in /usr/local/lib/python3.12/dist-packa
Requirement already satisfied: wrapt in /usr/local/lib/python3.12/dist-packages
```

```
import spacy # For general Natural Language Processing (NLP) tasks
from spacy import displacy # For visualizing Named Entity Recognition (NER) res
from transformers import pipeline, AutoTokenizer, AutoModelForTokenClassificati
import torch # Backend requirement for Hugging Face Transformers models
import pandas as pd # For data manipulation and creating DataFrames

print("Libraries imported successfully.")
```

```
Libraries imported successfully.
```

```
import spacy.cli

# 1. Download the en_core_web_sm spaCy model
# Check if the model is already downloaded to avoid re-downloading
try:
    nlp = spacy.load("en_core_web_sm")
    print("spaCy model 'en_core_web_sm' is already downloaded and loaded.")
except OSError:
    print("Downloading spaCy model 'en_core_web_sm'...")
    spacy.cli.download("en_core_web_sm")
    print("Model 'en_core_web_sm' downloaded.")
    # 2. Load the downloaded spaCy model
    nlp = spacy.load("en_core_web_sm")
    print("spaCy model 'en_core_web_sm' loaded successfully.")
```

```
spaCy model 'en_core_web_sm' is already downloaded and loaded.
```

```
sentences = [
    "The new AI model from Google DeepMind achieved a record-breaking score on
    "Lionel Messi scored a stunning hat-trick, leading Paris Saint-Germain to a
    "Economists predict a slight increase in interest rates following the centr
    "NASA's Perseverance rover successfully collected its first rock samples on
    "The government announced new legislation aimed at improving cybersecurity
]

print("Sample sentences created:")
for i, sentence in enumerate(sentences):
    print(f"Sentence {i+1}: {sentence}")
```

```
Sample sentences created:
Sentence 1: The new AI model from Google DeepMind achieved a record-breaking sco
Sentence 2: Lionel Messi scored a stunning hat-trick, leading Paris Saint-Germai
Sentence 3: Economists predict a slight increase in interest rates following the
Sentence 4: NASA's Perseverance rover successfully collected its first rock samp
Sentence 5: The government announced new legislation aimed at improving cybersec
```

```
print("\n--- spaCy NER Results ---")
for i, sentence in enumerate(sentences):
    doc = nlp(sentence)
    print(f"\nSentence {i+1}: {sentence}")
    if doc.ents:
        print("  Entities found:")
        for ent in doc.ents:
            print(f"    - Text: '{ent.text}', Label: '{ent.label_}'")
```

```
    else:
        print("  No entities found.")
```

```
--- spaCy NER Results ---

Sentence 1: The new AI model from Google DeepMind achieved a record-breaking sco
  Entities found:
    - Text: 'Google DeepMind', Label: 'ORG'

Sentence 2: Lionel Messi scored a stunning hat-trick, leading Paris Saint-Germai
  Entities found:
    - Text: 'Lionel Messi', Label: 'PERSON'
    - Text: 'Paris Saint-Germain', Label: 'FAC'
    - Text: 'the Champions League', Label: 'ORG'

Sentence 3: Economists predict a slight increase in interest rates following the
  No entities found.

Sentence 4: NASA's Perseverance rover successfully collected its first rock samp
  Entities found:
    - Text: 'NASA', Label: 'ORG'
    - Text: 'first', Label: 'ORDINAL'
    - Text: 'Mars', Label: 'LOC'

Sentence 5: The government announced new legislation aimed at improving cybersec
  No entities found.
```

```
spacy_ner_data = []

for sentence in sentences:
    doc = nlp(sentence)
    if doc.ents:
        for ent in doc.ents:
            spacy_ner_data.append({
                'Sentence': sentence,
                'Entity': ent.text,
                'Label': ent.label_
            })
    else:
        # Optionally, include sentences with no entities
        spacy_ner_data.append({
            'Sentence': sentence,
            'Entity': 'N/A',
            'Label': 'N/A'
        })

spacy_ner_df = pd.DataFrame(spacy_ner_data)

print("--- spaCy NER Results DataFrame ---")
print(spacy_ner_df)
```

```
--- spaCy NER Results DataFrame ---
                                            Sentence                Entity  \
0  The new AI model from Google DeepMind achieved...       Google DeepMind
1  Lionel Messi scored a stunning hat-trick, lead...          Lionel Messi
2  Lionel Messi scored a stunning hat-trick, lead...   Paris Saint-Germain
3  Lionel Messi scored a stunning hat-trick, lead...  the Champions League
```

```
4  Economists predict a slight increase in intere...                N/A
5  NASA's Perseverance rover successfully collect...               NASA
6  NASA's Perseverance rover successfully collect...              first
7  NASA's Perseverance rover successfully collect...               Mars
8  The government announced new legislation aimed...                N/A

     Label
0      ORG
1   PERSON
2      FAC
3      ORG
4      N/A
5      ORG
6  ORDINAL
7      LOC
8      N/A
```

```
print("\n--- Visualizing spaCy NER Results ---")
for i, sentence in enumerate(sentences):
    doc = nlp(sentence)
    print(f"\nSentence {i+1}:")
    if doc.ents:
        displacy.render(doc, style='ent', jupyter=True)
    else:
        print("  No entities to visualize.")
```

```
--- Visualizing spaCy NER Results ---

Sentence 1:
```

The new AI model from Google DeepMind **ORG** achieved a record-breaking score on the latest benchmark.

```
Sentence 2:
```

Lionel Messi **PERSON** scored a stunning hat-trick, leading Paris Saint-Germain **FAC** to a decisive victory in the Champions League **ORG** .

```
Sentence 3:
  No entities to visualize.

Sentence 4:
```

NASA **ORG** 's Perseverance rover successfully collected its first **ORDINAL** rock samples on Mars **LOC** , providing crucial data for future missions.

```
Sentence 5:
  No entities to visualize.
```

```
print("\n--- Loading Hugging Face NER Model ---")
# 1. Choose a suitable Hugging Face pretrained NER model
model_name = "dslim/bert-base-NER"

# 2. Load the tokenizer for the chosen model
```

```
tokenizer = AutoTokenizer.from_pretrained(model_name)
print(f"Tokenizer for '{model_name}' loaded successfully.")

# 3. Load the model for token classification
model = AutoModelForTokenClassification.from_pretrained(model_name)
print(f"Model '{model_name}' loaded successfully.")

# 4. Create an NER pipeline
hf_ner_pipeline = pipeline("ner", model=model, tokenizer=tokenizer, aggregation
print("Hugging Face NER pipeline created successfully.")
```

```
--- Loading Hugging Face NER Model ---
/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.py:94: UserW
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab (
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access publ
  warnings.warn(
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF
WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated req
```

config.json: 100% 829/829 [00:00<00:00, 74.9kB/s]

tokenizer_config.json: 100% 59.0/59.0 [00:00<00:00, 4.98kB/s]

vocab.txt: 213k/? [00:00<00:00, 6.91MB/s]

added_tokens.json: 100% 2.00/2.00 [00:00<00:00, 174B/s]

special_tokens_map.json: 100% 112/112 [00:00<00:00, 9.55kB/s]

```
Tokenizer for 'dslim/bert-base-NER' loaded successfully.
```

model.safetensors: 100% 433M/433M [00:05<00:00, 62.2MB/s]

Loading weights: 100% 199/199 [00:00<00:00, 549.27it/s, Materializing param=classifier.weight]

```
BertForTokenClassification LOAD REPORT from: dslim/bert-base-NER
Key                       | Status     |  |
--------------------------+------------+--+-
bert.pooler.dense.bias    | UNEXPECTED |  |
bert.pooler.dense.weight  | UNEXPECTED |  |

Notes:
- UNEXPECTED    :can be ignored when loading from different task/architecture; n
Model 'dslim/bert-base-NER' loaded successfully.
Hugging Face NER pipeline created successfully.
```

Task

```
print("\n--- Applying Hugging Face NER ---")
hf_ner_results = []

for i, sentence in enumerate(sentences):
    # Pass each sentence to the Hugging Face transformer NER pipeline
    # The pipeline returns a list of dictionaries, each representing an identified
```

```
    entities = hf_ner_pipeline(sentence)

    print(f"\nSentence {i+1}: {sentence}")
    if entities:
        print("  Entities found:")
        for entity in entities:
            # Extract word, entity label, and confidence score
            word = entity['word']
            entity_label = entity['entity']
            score = entity['score']
            print(f"    - Word: '{word}', Entity Label: '{entity_label}', Confidenc

            hf_ner_results.append({
                'Sentence': sentence,
                'Word': word,
                'Entity_Label': entity_label,
                'Confidence': score
            })
    else:
        print("  No entities found.")
        hf_ner_results.append({
            'Sentence': sentence,
            'Word': 'N/A',
            'Entity_Label': 'N/A',
            'Confidence': 'N/A'
        })
```

```
    hf_ner_results = []

    print("\n--- Hugging Face NER Results ---")
    for i, sentence in enumerate(sentences):
        print(f"\nSentence {i+1}: {sentence}")
        entities = hf_ner_pipeline(sentence)

        if entities:
            print("  Entities found:")
            for ent in entities:
                word = ent['word']
                entity_label = ent['entity_group'] # 'entity_group' is the aggregat
                score = ent['score']
                print(f"    - Word: '{word}', Label: '{entity_label}', Confidence:
                hf_ner_results.append({
                    'Sentence': sentence,
                    'Word': word,
                    'Entity_Label': entity_label,
                    'Confidence': score
                })
        else:
            print("  No entities found.")
            hf_ner_results.append({
                'Sentence': sentence,
```

```
            'Word': 'N/A',
            'Entity_Label': 'N/A',
            'Confidence': 'N/A'
        })
```

```
--- Hugging Face NER Results ---

Sentence 1: The new AI model from Google DeepMind achieved a record-breaking sco
  Entities found:
    - Word: 'AI', Label: 'MISC', Confidence: 0.9833
    - Word: 'Google DeepMind', Label: 'ORG', Confidence: 0.9862

Sentence 2: Lionel Messi scored a stunning hat-trick, leading Paris Saint-Germai
  Entities found:
    - Word: 'Lionel Messi', Label: 'PER', Confidence: 0.9993
    - Word: 'Paris Saint - Germain', Label: 'ORG', Confidence: 0.9988
    - Word: 'Champions League', Label: 'MISC', Confidence: 0.9986

Sentence 3: Economists predict a slight increase in interest rates following the
  No entities found.

Sentence 4: NASA's Perseverance rover successfully collected its first rock samp
  Entities found:
    - Word: 'NASA', Label: 'ORG', Confidence: 0.9988
    - Word: 'Perse', Label: 'MISC', Confidence: 0.5619
    - Word: '##verance', Label: 'ORG', Confidence: 0.3977
    - Word: 'Mars', Label: 'LOC', Confidence: 0.9986

Sentence 5: The government announced new legislation aimed at improving cybersec
  No entities found.
```

```
cleaned_hf_ner_results = []

i = 0
while i < len(hf_ner_results):
    current_entity = hf_ner_results[i]
    if current_entity['Word'] == 'N/A':
        cleaned_hf_ner_results.append(current_entity)
        i += 1
        continue

    # Start with the current entity's details
    combined_word = current_entity['Word']
    combined_label = current_entity['Entity_Label']
    combined_score = current_entity['Confidence']
    original_sentence = current_entity['Sentence']

    j = i + 1
    # Check for subsequent subword tokens that belong to the same entity
    while j < len(hf_ner_results) and hf_ner_results[j]['Sentence'] == original
        next_entity = hf_ner_results[j]
        # Hugging Face often uses '##' for subwords, and also splits words with
        # We consolidate if the current word ends with a hyphen or if the next
        # This heuristic might need refinement depending on the tokenizer.
        if next_entity['Word'].startswith('##') and next_entity['Entity_Label']
            combined_word += next_entity['Word'].replace('##', '')
```

```
                # Average confidence score or take the minimum/maximum
                combined_score = (combined_score + next_entity['Confidence']) / 2 #
                j += 1
            elif next_entity['Entity_Label'] == combined_label and next_entity['Con
                # This case handles splits like 'Perse' and 'verance' for 'Persever
                # We need a more robust way to check if they are part of the same o
                # For now, a simple concatenation and checking if the previous word
                # This is a simplification; a more robust approach involves charact
                # Given the current data, 'Perse' and '##verance' are a clearer cas
                # For 'Perse' and '##verance', the '##' check works.
                # Let's stick to the '##' for now for clearer consolidation.
                break # Stop combining if not a '##' subword
            else:
                break

        # Add the consolidated entity
        cleaned_hf_ner_results.append({
            'Sentence': original_sentence,
            'Word': combined_word,
            'Entity_Label': combined_label,
            'Confidence': combined_score
        })
        i = j # Move pointer to the next unconsolidated entity

    print("\n--- Cleaned Hugging Face NER Results (First 10 entries) ---")
    for i, result in enumerate(cleaned_hf_ner_results[:10]):
        confidence_str = f"{result['Confidence']:.4f}" if isinstance(result['Confid
        print(f"Entry {i+1}: Sentence: '{result['Sentence'][:50]}...', Word: '{resu
```

```
--- Cleaned Hugging Face NER Results (First 10 entries) ---
Entry 1: Sentence: 'The new AI model from Google DeepMind achieved a r...', Word
Entry 2: Sentence: 'The new AI model from Google DeepMind achieved a r...', Word
Entry 3: Sentence: 'Lionel Messi scored a stunning hat-trick, leading ...', Word
Entry 4: Sentence: 'Lionel Messi scored a stunning hat-trick, leading ...', Word
Entry 5: Sentence: 'Lionel Messi scored a stunning hat-trick, leading ...', Word
Entry 6: Sentence: 'Economists predict a slight increase in interest r...', Word
Entry 7: Sentence: 'NASA's Perseverance rover successfully collected i...', Word
Entry 8: Sentence: 'NASA's Perseverance rover successfully collected i...', Word
Entry 9: Sentence: 'NASA's Perseverance rover successfully collected i...', Word
Entry 10: Sentence: 'NASA's Perseverance rover successfully collected i...', Wor
```

```
cleaned_hf_ner_results = []

i = 0
while i < len(hf_ner_results):
    current_entity = hf_ner_results[i]

    # Handle 'N/A' entries for sentences with no entities
    if current_entity['Word'] == 'N/A':
        cleaned_hf_ner_results.append(current_entity)
        i += 1
        continue

    combined_word = current_entity['Word']
    combined_label = current_entity['Entity_Label']
    combined_score = current_entity['Confidence']
```

```
        original_sentence = current_entity['Sentence']

        start_idx = i
        j = i + 1

        # Loop to check for subsequent subword tokens or multi-token entities
        while j < len(hf_ner_results) and hf_ner_results[j]['Sentence'] == original_s
            next_entity = hf_ner_results[j]

            # More robust logic for consolidating entities:
            # 1. If next word starts with '##' and labels are consistent (or can be r
            # 2. Heuristic for multi-token entities that might not have '##' but repr

            # Case 1: Subword tokens using '##'
            if next_entity['Word'].startswith('##'):
                # If the labels are different but it's clearly a subword, prioritize
                # and potentially updating the label based on the most dominant or fi
                combined_word += next_entity['Word'].replace('##', '')
                # Average confidence score across merged parts
                combined_score = (combined_score + next_entity['Confidence']) / 2
                # If the original label was generic (MISC) and next is specific, upda
                if combined_label == 'MISC' and next_entity['Entity_Label'] != 'MISC'
                    combined_label = next_entity['Entity_Label']
                j += 1
            # Case 2: Multi-word entities that are not subwords but are grouped by th
            # Example: 'Paris Saint - Germain' was already aggregated by the pipeline
            # This block might be redundant if 'aggregation_strategy="simple"' handle
            # but could be useful for further refinement if specific multi-word entit
            elif next_entity['Entity_Label'] == combined_label and not next_entity['W
                # This condition is tricky as it might over-merge. Only merge if very
                # For example, if 'New York' is split into ['New', 'York'] with same
                # For this dataset, 'aggregation_strategy="simple"' handles this well
                break # For now, let's rely on 'aggregation_strategy="simple"' for no
            else:
                break

        # Add the consolidated entity
        cleaned_hf_ner_results.append({
            'Sentence': original_sentence,
            'Word': combined_word,
            'Entity_Label': combined_label,
            'Confidence': combined_score
        })
        i = j # Move pointer to the next unconsolidated entity

    print("\n--- Cleaned Hugging Face NER Results (First 10 entries) ---")
    for i, result in enumerate(cleaned_hf_ner_results[:10]):
        confidence_str = f"{result['Confidence']:.4f}" if isinstance(result['Confider
        print(f"Entry {i+1}: Sentence: '{result['Sentence'][:50]}...', Word: '{result
```

```
--- Cleaned Hugging Face NER Results (First 10 entries) ---
Entry 1: Sentence: 'The new AI model from Google DeepMind achieved a r...', Word
Entry 2: Sentence: 'The new AI model from Google DeepMind achieved a r...', Word
Entry 3: Sentence: 'Lionel Messi scored a stunning hat-trick, leading ...', Word
Entry 4: Sentence: 'Lionel Messi scored a stunning hat-trick, leading ...', Word
Entry 5: Sentence: 'Lionel Messi scored a stunning hat-trick, leading ...', Word
```

```
Entry 6: Sentence: 'Economists predict a slight increase in interest r...', Word
Entry 7: Sentence: 'NASA's Perseverance rover successfully collected i...', Word
Entry 8: Sentence: 'NASA's Perseverance rover successfully collected i...', Word
Entry 9: Sentence: 'NASA's Perseverance rover successfully collected i...', Word
Entry 10: Sentence: 'The government announced new legislation aimed at ...', Wor
```